AGRITECH

Beginner Guide: Clone · Install · Run · Present

Clean step-by-step instructions for first-time users

Repository: `https://github.com/crispyvicky/agritech.git`

Demo images folder: `AgriSync\DEMO_TESTED_IMAGES`

1. Before You Start (What You Need)

This guide is for beginners. Follow the steps in order. You will clone the project from GitHub, install dependencies, start two terminals (backend + frontend), open the website, and upload tested demo images.

Install these once on your PC:

- Git — https://git-scm.com/downloads
- Python 3.10 or 3.11 (64-bit) — https://www.python.org/downloads/ (tick “Add Python to PATH”)
- Node.js LTS (includes npm) — https://nodejs.org/
- Google Chrome or Microsoft Edge

Recommended: 8 GB RAM or more (TensorFlow needs memory).

2. How to Clone This Repository

2.1 Open a terminal

On Windows: press Win key, type PowerShell, open Windows PowerShell. Or open Terminal inside VS Code / Cursor.

2.2 Go to your Desktop (or any folder you like)

```
cd $env:USERPROFILE\Desktop
```

2.3 Clone AgriSync

```
git clone https://github.com/crispyvicky/agritech.git
cd AgriSync
```

After this you should see folders such as backend, frontend, DEMO_TESTED_IMAGES, and docs.

If you already have the project (for example on Desktop\agri\AgriSync), skip cloning and just cd into that AgriSync folder.

3. Install (First Time Only)

3.1 Backend (Python + AI)

Create a virtual environment and install packages. This may take several minutes because TensorFlow is large.

```
cd backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
pip install reportlab
cd ..
```

If Activate.ps1 is blocked, run once: Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

3.2 Frontend (React)

```
cd frontend
npm install
cd ..
```

Wait until npm finishes without red errors. You only need this once (or after package changes).

4. How to Run Cleanly (Every Demo)

You need TWO terminals. Keep both open while you use the website.

4.1 Terminal 1 — Start Backend (port 8000)

```
cd path\to\AgriSync\backend
$env:PORT="8000"
$env:PYTHONUTF8="1"
.\.venv\Scripts\python.exe start.py
```

Wait until you see the API starting / Uvicorn running. First start may load models and take longer. Test in browser: http://localhost:8000/health — should say API is running.

4.2 Terminal 2 — Start Frontend (port 3000)

```
cd path\to\AgriSync\frontend
npm start
```

Browser should open http://localhost:3000. If not, open that URL yourself.

4.3 Stop cleanly when finished

In each terminal press Ctrl+C. Do not close only the browser if you want to free the ports.

5. How to Present (Simple Script)

Say this in about 2–3 minutes while clicking:

1. AgriTech is a web app that helps detect plant leaf diseases and classify soil from photos.
2. It uses a Machine Learning algorithm: EfficientNetB0 (CNN) embeddings + cosine similarity matching.
3. Open Plant Disease Detection → upload a leaf → Analyze → show class, confidence, HEALTHY/DISEASED, and treatment tip.
4. Open Soil Detection → upload soil → Predict → show soil type and which crops can grow + care tips.
5. Optional: swap images (soil into plant / leaf into soil) to show the “upload correct image only” message.
6. Mention demo images are pre-tested in DEMO_TESTED_IMAGES for accurate live results.

5.1 What the ML algorithm is (one sentence for viva)

“We extract a deep feature vector from the photo using EfficientNetB0 pretrained on ImageNet, then match it to the nearest class prototype with cosine similarity, and finally attach treatment or crop advice.”

6. Which Images to Upload (Use These)

For the most accurate demo results, upload images from this folder only:

```
AgriSync\DEMO_TESTED_IMAGES\
  plant_disease\
  soil\
  README.txt
```

Full path on a typical machine after clone:

```
...\AgriSync\DEMO_TESTED_IMAGES\plant_disease\
...\AgriSync\DEMO_TESTED_IMAGES\soil\
```

6.1 Plant Disease Detection — upload from plant_disease\

File	What you should see
01_tomato_late_blight.jpg	Tomato late blight · DISEASED · treatment tip
02_apple_leaf_healthy.jpg	Apple leaf · HEALTHY
03_corn_rust_leaf.jpg	Corn rust leaf · DISEASED

6.2 Soil Detection — upload from soil\

File	What you should see
01_alluvial_soil.jpg / 02_...	Alluvial soil + crops / care
03_black_soil.jpg / 04_...	Black Soil + crops / care
05_clay_soil.jpg	Clay soil + crops / care
06_red_soil.jpg	Red soil + crops / care

6.3 Wrong-image demo (optional, impressive)

- Put a soil photo into Plant Disease → should say upload plant leaf only.
- Put a leaf photo into Soil Detection → should say upload soil only.

6.4 Can I upload my own photos?

Yes — external images can work if they are clear close-up leaf photos for the plant tool, or clear soil surface photos for the soil tool. For a graded presentation, prefer DEMO_TESTED_IMAGES so results stay reliable.

7. Quick Visual Preview of Demo Images



Plant demo: 01_tomato_late_blight.jpg



Plant demo: 02_apple_leaf_healthy.jpg



Soil demo: 01_alluvial_soil.jpg



Soil demo: 03_black_soil.jpg

8. Troubleshooting (Beginner Fixes)

Port already in use

Close old terminals, or stop the process using port 8000/3000, then start again.

npm start fails

Delete frontend\node_modules and run npm install again inside frontend.

Backend import / TensorFlow error

Activate .venv and re-run pip install -r requirements.txt.

Frontend cannot predict

Confirm backend health at http://localhost:8000/health first.

Weird plant result / low confidence

Use DEMO_TESTED_IMAGES\plant_disease files; use a clear single-leaf photo.

Prediction slow first time

Normal — models load on first request. Wait once, then retry.

9. Related Documents

- College project report: docs\AgriTech final.pdf
- Short run notes: RUN_PRESENTATION.md
- Image list: DEMO_IMAGES.md and DEMO_TESTED_IMAGES\README.txt

Checklist before presentation: Backend running · Frontend open · DEMO_TESTED_IMAGES ready · Browser zoom comfortable for projector.

— End of Setup & Demo Guide —